Time Series Forecasting and Anomaly Detection Internship

Interview Preparation Guide

Newcastle University - School of Engineering
Duration: 60 hours (3 hours/week, February-July 2026)
Supervisor: Dr. Ekin Can Erkus (Senior Research Associate, Microsystems Group)

Overview

This internship focuses on time series forecasting and anomaly detection across engineering and scientific domains (energy systems, industrial processes, health monitoring). The project examines classical forecasting models, machine learning methods, and deep learning approaches, emphasizing research thinking and scientific reporting with potential for academic publication.

Key Focus Areas: Seasonal variation, noise handling, sudden changes, explainability, comparative research

---

Part 1: Behavioral Questions (25)

Research & Analytical Thinking (5)

1. Describe a research project where you compared multiple approaches. How did you ensure fair comparison?
2. Experience implementing a published paper or algorithm?
3. Time you identified patterns in noisy data
4. How do you balance exploration vs exploitation in research?
5. Experience with experimental design and hypothesis testing

Technical Problem-Solving (5)

6. Debugging a mathematical/statistical model that wasn't converging
7. Optimizing algorithm performance (time/memory)
8. Trade-offs between model complexity and interpretability
9. Handling contradictory results from different methods
10. Failed experiment - how did you troubleshoot?

Learning & Independence (5)

11. Self-learning advanced mathematical concepts

12. Experience with scientific computing libraries (NumPy, SciPy, scikit-learn)
13. Working independently on technical project with weekly check-ins
14. Understanding a complex algorithm from literature
15. Bridging gap between theory and implementation

Communication & Documentation (5)

16. Explaining statistical concepts to non-statisticians
17. Documenting experimental methodology for reproducibility
18. Creating visualizations to communicate results
19. Writing technical reports or papers
20. Presenting research findings (academic vs industry audience)

Time Management & Research Skills (5)

21. Managing a 6-month research project with weekly deliverables
22. Balancing exploration with meeting deadlines
23. Prioritizing which experiments to run
24. Literature review strategy
25. Handling unexpected negative results

Part 2: Technical Questions (30)

A: Time Series Fundamentals & Mathematics (8)

Q1: Stationarity Test

```
import numpy as np
from statsmodels.tsa.stattools import adfuller

ts = load_time_series()
# Test for stationarity
```

What is stationarity? Why is it important? How would you test for it and make a series stationary?

Q2: Autocorrelation & Partial Autocorrelation Explain the difference between ACF and PACF. Given these plots, how would you determine the order of an ARIMA model?

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
# Plot shows ACF cuts off at lag 2, PACF decays gradually
```

Q3: Decomposition Math A time series can be decomposed as: $Y(t) = T(t) + S(t) + R(t)$

Where T=trend, S=seasonal, R=residual.

a) What's the difference between additive and multiplicative decomposition?

b) When would you use each?

c) Implement seasonal decomposition from scratch (don't use libraries)

Q4: Moving Average Calculation

```
data = [10, 12, 15, 18, 14, 16, 20, 22, 19, 25]
# Calculate 3-period simple moving average
# Then calculate exponential moving average with  =0.3
```

Show the formulas and implement both.

Q5: Differencing

```
ts = pd.Series([100, 105, 110, 115, 118, 125, 130])
# Make stationary using differencing
```

What is differencing? When do you use first vs second order? How does it relate to integration in ARIMA?

Q6: Fourier Analysis A time series has hourly data with strong daily (24h) and weekly (168h) patterns.

a) How would you use FFT to identify dominant frequencies?

b) Write code to extract the two strongest periodic components

c) What are the limitations of Fourier analysis for non-stationary data?

Q7: White Noise Test

```
residuals = model.predict(X) - y
# Test if residuals are white noise
```

What is white noise? Why is it important? How would you test for it? (Ljung-Box test)

Q8: Seasonal Strength Calculation Given a decomposed time series, calculate: - Trend strength: FT = 1 - Var(R) / Var(T + R) - Seasonal strength: FS = 1 - Var(R) / Var(S + R)

Implement these formulas and interpret values between 0 and 1.

B: Classical Forecasting Models (6)

Q9: ARIMA Model Selection You have time series with trend and seasonality.

a) What do the (p,d,q) parameters mean in ARIMA(p,d,q)?

b) How would you select them? (AIC, BIC, cross-validation)

c) Implement ARIMA from scratch for p=1, d=1, q=1

Q10: Exponential Smoothing

```
# Simple Exponential Smoothing
def ses(data, alpha):
    forecast = [data[0]]
    for i in range(1, len(data)):
        forecast.append(alpha * data[i-1] + (1 - alpha) * forecast[-1])
    return forecast
```

This is wrong. Fix it. Then extend to Holt's linear trend method (double exponential smoothing).

Q11: SARIMA Explain SARIMA(p,d,q)(P,D,Q)m notation. - What are the seasonal components (P,D,Q)? - What is m? - For hourly data with daily seasonality, what would m be? - When would you use SARIMA vs ARIMA?

Q12: AR vs MA

```
# AR(1): y_t =  ·y_{t-1} + _t
# MA(1): y_t = _t +  · _{t-1}
```

Explain the conceptual difference. Given ACF/PACF patterns, how distinguish AR from MA process?

Q13: Forecast Intervals Your model predicts y_t = 100. Calculate 95% prediction interval given: - Residual standard error = 5 - h-step ahead forecast (h=3) - Assume residuals are normal

Show formula and calculation.

Q14: Model Diagnostics After fitting ARIMA, what diagnostic checks should you perform? - Residual plots - ACF of residuals
- Ljung-Box test - Normality tests

Write code to automate these checks.

C: Machine Learning for Time Series (8)

Q15: Train-Test-Validation Split

```
# Wrong approach
X_train, X_test = train_test_split(X, y, test_size=0.2, shuffle=True)
```

Why wrong? Implement proper time series split with: - Training: 60%
- Validation: 20%
- Test: 20% - Ensure no data leakage

Q16: Feature Engineering Given timestamp and value, create 10 features for ML model: - Temporal features (hour, day, month, etc.) - Lag features - Rolling statistics - Fourier features

Implement comprehensively.

Q17: Gradient Boosting for Time Series

```
from xgboost import XGBRegressor

model = XGBRegressor()
model.fit(X_train, y_train)
pred = model.predict(X_test)
```

What's missing for proper time series forecasting? How create sliding windows? Multi-step ahead forecasting?

Q18: LSTM Architecture Design Design LSTM for multi-step forecasting (predict next 24 hours).

a) Input shape for sequence length=48, features=5?

b) Difference between return_sequences=True vs False?

c) How implement recursive vs direct multi-step forecasting?

Q19: Attention Mechanism Explain self-attention in Transformers for time series. - How does attention weight calculation work? (Q, K, V matrices) - Why effective for long sequences? - Computational complexity vs LSTM?

Q20: Hyperparameter Tuning Design cross-validation strategy for hyperparameter tuning in time series: - Time Series Split CV - Grid Search vs Random Search vs Bayesian Optimization - What metrics to optimize?

```
from sklearn.model_selection import TimeSeriesSplit
# Implement proper CV for time series
```

Q21: Ensemble Methods You have 3 models: - ARIMA: RMSE=10, captures trend well - LSTM: RMSE=12, captures patterns well - XGBoost: RMSE=11, handles non-linearities

How would you ensemble them? Simple average, weighted average, stacking?

Q22: Transfer Learning You've trained a forecasting model on one time series. How adapt to new series with limited data? - Fine-tuning approach - Feature extraction - Domain adaptation techniques

---

D: Anomaly Detection (6)

Q23: Statistical Anomaly Detection

```
# Detect anomalies using z-score
def detect_anomalies(data, threshold=3):
    z_scores = (data - data.mean()) / data.std()
    return abs(z_scores) > threshold
```

What's the problem with this approach? Implement better methods: - Rolling z-score - IQR method - Handling seasonality

---

Q24: Residual-Based Detection After fitting ARIMA, use residuals for anomaly detection.

a) Why are residuals useful?

b) How determine threshold? (statistical vs adaptive)

c) Difference between point anomalies vs contextual anomalies?

Q25: Autoencoder for Anomalies

```
# Autoencoder architecture
encoder = [128, 64, 32]
decoder = [32, 64, 128]
```

How use reconstruction error for anomaly detection? - Training on normal data only - Threshold selection (percentile vs statistical) - Handling temporal dependencies (LSTM-Autoencoder)

Q26: Isolation Forest Explain Isolation Forest algorithm: - How does it work? (tree-based isolation) - Why effective for anomalies? - Advantages over other methods? - Implement contamination parameter selection

---

Q27: Time Series Specific Anomalies Classify these anomaly types: 1. Single point spike 2. Level shift 3. Trend change 4. Seasonal pattern disruption

How would you detect each differently?

Q28: Evaluation Metrics For anomaly detection with imbalanced data (1% anomalies):

a) Why is accuracy misleading?

b) Calculate precision, recall, F1 from confusion matrix

c) What is ROC-AUC and why useful here?

d) Implement precision-recall curve

E: Implementation & Research Skills (2)

Q29: Reproducibility Ensure your experiments are reproducible: - Random seeds - Version control (Git) - Environment management (require-

ments.txt, conda) - Experiment tracking (parameters, metrics, artifacts)

Write a template experiment script.

Q30: Comparative Analysis You must compare 5 methods: ARIMA, ETS, Prophet, LSTM, Transformer.

a) Design experimental framework

b) Fair comparison considerations (same train/test, same metrics)

c) Statistical significance testing (Diebold-Mariano test)

d) Result visualization and reporting

Part 3: Sample Answers

Behavioral Answers

Q1: Comparing Multiple Approaches "In my third-year project on traffic flow prediction, I compared four methods: ARIMA, Prophet, LSTM, and GRU.

To ensure fairness, I: 1. Used identical train/test splits (chronological, 80/20) 2. Same evaluation metrics (RMSE, MAE, MAPE) 3. Similar hyperparameter tuning budgets (50 trials each via Optuna) 4. Same computational resources (local GPU, 1 hour max per model) 5. Identical data preprocessing

Created systematic comparison table showing: - Accuracy metrics - Training time - Inference speed - Memory requirements - Interpretability score (subjective 1-5)

ARIMA was fastest and most interpretable but least accurate. LSTM achieved best accuracy but was slowest. Documented all details in Jupyter notebook with versioned data.

Key learning: Fair comparison requires controlling all variables except the method itself. Also, 'best' depends on constraints—production needs might favor ARIMA's speed over LSTM's accuracy."

Why strong: Systematic approach, specific details, considers multiple dimensions, acknowledges trade-offs.

Q11: Self-Learning Advanced Math "When learning LSTM mathematics for my neural networks module, the concepts were abstract. I used multi-pronged approach:

1. **Theory first:** Read original Hochreiter & Schmidhuber paper (1997), then modern tutorials (colah's blog on LSTMs)
2. **Mathematical breakdown:** Wrote out all equations by hand:
   - Forget gate: f_t = (W_f · [h_{t-1}, x_t] + b_f)
   - Input gate, output gate, cell state updates
   - Traced through one complete timestep manually
3. **Numerical example:** Used tiny 2x2 matrices, computed forward pass step-by-step in Excel to see hidden states evolve
4. **Implementation from scratch:** Built LSTM in NumPy (no PyTorch), debugged gradients against numerical gradients
5. **Visualization:** Plotted gate activations over time to understand what each gate was doing

What clicked: Implementing from scratch forced me to understand every operation. Seeing actual numbers flowing through gates made abstract equations concrete.

Now when I use PyTorch LSTM, I know exactly what's happening under the hood—helpful for debugging weird training behavior."

Why strong: Structured learning approach, goes beyond tutorials to fundamentals, active learning through implementation, connects theory to practice.

Q16: Explaining Statistical Concepts "Had to explain time series forecasting to marketing team for sales predictions.

Avoided jargon like 'ARIMA(2,1,2)' or 'autocorrelation.' Instead used:

Trend: 'The general direction—are sales going up or down over months?' **Seasonality:** 'Patterns that repeat regularly—like Christmas sales spike every December' **Noise:** 'Random daily variation we can't predict'

Used visual analogies: - Showed decomposition plot: 'We break the messy signal into these clean patterns' - For forecasting: 'Based on last 3 years, here's what we expect next quarter—with confidence bands like weather forecast (70% chance sales between X and Y)'

For model selection, explained as: 'We tested 4 methods. This one (ARIMA) is simple and fast but less accurate. This one (LSTM) is very accurate but takes longer to update. Given your need for weekly updates, we recommend ARIMA—95% as accurate, 10x faster.'

Result: They understood enough to ask good questions like 'What if we have a new product launch?' (I explained limitations of historical methods with structural breaks).

Key lesson: Use their domain language, visual examples, and focus on implications not mechanisms."

Why strong: Audience awareness, concrete analogies, visual communication, connects to their needs.

Technical Answers

Q1: Stationarity

What is stationarity?

A time series is stationary if its statistical properties don't change over time: - Constant mean: E[Y_t] = (doesn't vary with t) - Constant variance: Var[Y_t] = ^{2} (doesn't vary with t) - Constant autocovariance: Cov[Y_t, Y_{t+k}] depends only on k, not t

Why important? - Many statistical methods (ARIMA, VAR) assume stationarity - Easier to model—patterns don't change - Statistical properties are predictable

Testing for stationarity:

```
import numpy as np
import pandas as pd
from statsmodels.tsa.stattools import adfuller, kpss

def test_stationarity(ts, print_results=True):
    """
    Test time series for stationarity using:
    1. ADF test (Augmented Dickey-Fuller)
    2. KPSS test
    3. Visual inspection
    """

    # 1. ADF Test
    # H0: Series has unit root (non-stationary)
    # H1: Series is stationary
    adf_result = adfuller(ts, autolag='AIC')

    adf_stat = adf_result[0]
    adf_pvalue = adf_result[1]
    adf_critical = adf_result[4]

    # 2. KPSS Test
    # H0: Series is stationary
    # H1: Series has unit root (non-stationary)
```

```
kpss_result = kpss(ts, regression='ct', nlags='auto')

kpss_stat = kpss_result[0]
kpss_pvalue = kpss_result[1]
kpss_critical = kpss_result[3]

if print_results:
    print("=" * 60)
    print("STATIONARITY TESTS")
    print("=" * 60)

    print("\n1. Augmented Dickey-Fuller Test:")
    print(f"   ADF Statistic: {adf_stat:.4f}")
    print(f"   p-value: {adf_pvalue:.4f}")
    print(f"   Critical Values:")
    for key, value in adf_critical.items():
        print(f"      {key}: {value:.4f}")

    if adf_pvalue < 0.05:
        print("    ADF: Series is stationary (reject H0)")
    else:
        print("    ADF: Series is non-stationary (fail to reject H0)")

    print("\n2. KPSS Test:")
    print(f"   KPSS Statistic: {kpss_stat:.4f}")
    print(f"   p-value: {kpss_pvalue:.4f}")
    print(f"   Critical Values:")
    for key, value in kpss_critical.items():
        print(f"      {key}: {value:.4f}")

    if kpss_pvalue > 0.05:
        print("    KPSS: Series is stationary (fail to reject H0)")
    else:
        print("    KPSS: Series is non-stationary (reject H0)")

    print("\n3. Overall Conclusion:")
    if adf_pvalue < 0.05 and kpss_pvalue > 0.05:
        print("   → Series is STATIONARY")
    elif adf_pvalue > 0.05 and kpss_pvalue < 0.05:
        print("   → Series is NON-STATIONARY")
    else:
        print("   → Results are INCONCLUSIVE (apply transformations)")
    print("=" * 60)

return {
    'adf_stat': adf_stat,
```

```
        'adf_pvalue': adf_pvalue,
        'kpss_stat': kpss_stat,
        'kpss_pvalue': kpss_pvalue,
        'is_stationary': (adf_pvalue < 0.05) and (kpss_pvalue > 0.05)
    }

# Making series stationary
def make_stationary(ts):
    """Transform non-stationary series to stationary"""

    # Method 1: Differencing
    ts_diff = ts.diff().dropna()

    # Method 2: Log transform then difference (for multiplicative trends)
    ts_log = np.log(ts + 1)  # +1 to avoid log(0)
    ts_log_diff = ts_log.diff().dropna()

    # Method 3: Detrending
    from scipy import signal
    ts_detrended = signal.detrend(ts)

    return {
        'differenced': ts_diff,
        'log_differenced': ts_log_diff,
        'detrended': ts_detrended
    }

# Example usage
ts = pd.Series([100, 105, 112, 118, 125, 133, 142, 150])

# Test original
test_stationarity(ts)

# Make stationary
ts_stationary = make_stationary(ts)

# Test differenced series
print("\n\nAfter differencing:")
test_stationarity(ts_stationary['differenced'])
```

Key insights: - Use both ADF and KPSS (complementary tests) - ADF tests for unit root, KPSS tests for stationarity - Differencing is most common transformation - Order of integration = number of differences needed

---

Q3: Decomposition from Scratch

Additive vs Multiplicative:

Additive: $Y(t) = T(t) + S(t) + R(t)$ - Use when seasonal variation is constant over time - Seasonal amplitude doesn't depend on trend level

Multiplicative: $Y(t) = T(t) \times S(t) \times R(t)$ - Use when seasonal variation increases with trend - Common in economic/financial data - Can be converted to additive via log: $\log(Y) = \log(T) + \log(S) + \log(R)$

Implementation:

```
import numpy as np
import pandas as pd
from scipy import signal

def seasonal_decompose_additive(ts, period=12):
    """
    Additive decomposition from scratch

    Steps:
    1. Extract trend using centered moving average
    2. Detrend to get seasonal + noise
    3. Average seasonal pattern over multiple cycles
    4. Residual = original - trend - seasonal
    """

    # 1. Trend: Centered moving average
    # Use period-length window, centered
    if period % 2 == 0:  # Even period
        # Need two-step averaging for centering
        ma1 = ts.rolling(window=period, center=False).mean()
        trend = ma1.rolling(window=2, center=False).mean().shift(-1)
    else:  # Odd period
        trend = ts.rolling(window=period, center=True).mean()

    # 2. Detrended = Original - Trend
    detrended = ts - trend

    # 3. Seasonal component
    # Average detrended values for each season
    seasonal = np.zeros(len(ts))

    for i in range(period):
        # Get all observations for this season (i, i+period, i+2*period, ...)
        season_vals = detrended[i::period].dropna()
        season_avg = season_vals.mean()

        # Assign to all positions of this season
```

```
        seasonal[i::period] = season_avg

    # Ensure seasonal component sums to zero (constraint)
    seasonal = seasonal - seasonal.mean()
    seasonal = pd.Series(seasonal, index=ts.index)

    # 4. Residual = Original - Trend - Seasonal
    residual = ts - trend - seasonal

    return {
        'observed': ts,
        'trend': trend,
        'seasonal': seasonal,
        'residual': residual
    }

def seasonal_decompose_multiplicative(ts, period=12):
    """
    Multiplicative decomposition: Y = T × S × R

    Convert to additive via log:
    log(Y) = log(T) + log(S) + log(R)
    """

    # Take log (handle zeros/negatives)
    ts_log = np.log(ts + 1e-10)

    # Apply additive decomposition to log
    decomp_log = seasonal_decompose_additive(ts_log, period)

    # Convert back via exp
    return {
        'observed': ts,
        'trend': np.exp(decomp_log['trend']),
        'seasonal': np.exp(decomp_log['seasonal']),
        'residual': np.exp(decomp_log['residual'])
    }

# Example
np.random.seed(42)
time = np.arange(100)

# Additive example: constant seasonal amplitude
trend = 0.5 * time + 50
seasonal = 10 * np.sin(2 * np.pi * time / 12)
noise = np.random.normal(0, 2, 100)
```

```
ts_additive = pd.Series(trend + seasonal + noise)

decomp_add = seasonal_decompose_additive(ts_additive, period=12)

print("Additive Decomposition:")
print(f"Trend range: [{decomp_add['trend'].min():.2f}, {decomp_add['trend'].max():.2f}]")
print(f"Seasonal range: [{decomp_add['seasonal'].min():.2f}, {decomp_add['seasonal'].max():
print(f"Residual std: {decomp_add['residual'].std():.2f}")

# Multiplicative example: seasonal amplitude grows with trend
trend_mult = np.exp(0.01 * time)
seasonal_mult = 1 + 0.2 * np.sin(2 * np.pi * time / 12)
noise_mult = 1 + np.random.normal(0, 0.05, 100)
ts_multiplicative = pd.Series(trend_mult * seasonal_mult * noise_mult)

decomp_mult = seasonal_decompose_multiplicative(ts_multiplicative, period=12)
```

When to use each: - **Additive:** Seasonal variation independent of level (temperature, sales units with stable promotions) - **Multiplicative:** Seasonal variation proportional to level (revenue, exponential growth processes)

Test: Plot original series. If seasonal swings get larger as trend increases → multiplicative. If constant → additive.

Q9: ARIMA Model Selection

ARIMA(p,d,q) parameters:

- **p (AR order):** Number of lagged observations to include
  - AR(p): y_t = _{t-1} + _{t-2} + … + _{t-p} + _t
- **d (Integration order):** Number of differences to make series stationary
  - d=0: no differencing
  - d=1: first difference (y_t - y_{t-1})
  - d=2: second difference
- **q (MA order):** Number of lagged forecast errors
  - MA(q): y_t = _t + _{t-1} + _{t-2} + … + _{t-q}

Selection strategies:

1. Visual inspection (ACF/PACF): - AR(p): PACF cuts off at lag p, ACF decays - MA(q): ACF cuts off at lag q, PACF decays - ARMA(p,q): Both decay

2. Information Criteria:

```
from statsmodels.tsa.arima.model import ARIMA
from itertools import product

def select_arima_order(ts, max_p=3, max_d=2, max_q=3):
    """
```

```
    Select ARIMA(p,d,q) using AIC/BIC
    """

    best_aic = np.inf
    best_bic = np.inf
    best_order_aic = None
    best_order_bic = None

    results = []

    # Grid search over parameter space
    for p in range(max_p + 1):
        for d in range(max_d + 1):
            for q in range(max_q + 1):
                try:
                    model = ARIMA(ts, order=(p, d, q))
                    fitted = model.fit()

                    aic = fitted.aic
                    bic = fitted.bic

                    results.append({
                        'order': (p, d, q),
                        'AIC': aic,
                        'BIC': bic
                    })

                    if aic < best_aic:
                        best_aic = aic
                        best_order_aic = (p, d, q)

                    if bic < best_bic:
                        best_bic = bic
                        best_order_bic = (p, d, q)

                except:
                    continue

    results_df = pd.DataFrame(results).sort_values('AIC')

    print(f"Best order by AIC: {best_order_aic} (AIC={best_aic:.2f})")
    print(f"Best order by BIC: {best_order_bic} (BIC={best_bic:.2f})")

    return results_df, best_order_aic, best_order_bic

# AIC = -2log(L) + 2k (penalizes complexity)
```

```
# BIC = -2log(L) + k*log(n) (stronger penalty)
# where L=likelihood, k=parameters, n=observations
```

3. Cross-validation:

```
from sklearn.model_selection import TimeSeriesSplit

def cv_arima(ts, order, n_splits=5):
    """Time series cross-validation for ARIMA"""

    tscv = TimeSeriesSplit(n_splits=n_splits)
    errors = []

    for train_idx, test_idx in tscv.split(ts):
        train = ts.iloc[train_idx]
        test = ts.iloc[test_idx]

        model = ARIMA(train, order=order)
        fitted = model.fit()

        forecast = fitted.forecast(steps=len(test))
        mse = ((test - forecast) ** 2).mean()
        errors.append(mse)

    return np.mean(errors), np.std(errors)
```

Simple ARIMA(1,1,1) implementation:

```
def arima_111(ts):
    """
    ARIMA(1,1,1) from scratch

    Model: (1 -  B)(1 - B)y_t = (1 +  B) _t
    where B is backshift operator

    Simplified: y_t - y_{t-1} =  (y_{t-1} - y_{t-2}) +  _t +  _{t-1}
    """

    # Step 1: First difference (d=1)
    y_diff = np.diff(ts)

    # Step 2: Estimate   and   using method of moments or MLE
    # For simplicity, use least squares approximation

    # Create lagged arrays
    y_t = y_diff[2:]
    y_lag1 = y_diff[1:-1]
    y_lag2 = y_diff[:-2]
```

```
    # Initial estimate of   from autocorrelation
    phi = np.corrcoef(y_t, y_lag1)[0, 1]

    # Iterate to refine (simplified - full MLE would use optimization)
    for iteration in range(10):
        # Compute residuals
        residuals = y_t - phi * y_lag1

        # Estimate   from residual autocorrelation
        theta = np.corrcoef(residuals[1:], residuals[:-1])[0, 1]

        # Refine
        phi = (np.sum(y_t * y_lag1) - theta * np.sum(residuals[:-1] * y_lag1[1:])) / np.sum(

    return {'phi': phi, 'theta': theta}
```

Best practice: Start with auto.arima equivalent (iterate through reasonable ranges), validate with cross-validation.

Q16: Comprehensive Feature Engineering

```
import pandas as pd
import numpy as np
from scipy import fft

def create_time_series_features(df, timestamp_col='timestamp', value_col='value',
                                lag_periods=[1, 2, 3, 7, 14, 30],
                                rolling_windows=[3, 7, 14, 30]):
    """
    Create comprehensive feature set for time series ML

    Returns: DataFrame with 50+ features
    """

    df = df.copy()
    df[timestamp_col] = pd.to_datetime(df[timestamp_col])
    df = df.sort_values(timestamp_col).reset_index(drop=True)

    features = pd.DataFrame(index=df.index)

    # ============ 1. TEMPORAL FEATURES ============

    # Basic datetime components
    features['year'] = df[timestamp_col].dt.year
    features['month'] = df[timestamp_col].dt.month
```

```
features['day'] = df[timestamp_col].dt.day
features['dayofweek'] = df[timestamp_col].dt.dayofweek
features['dayofyear'] = df[timestamp_col].dt.dayofyear
features['hour'] = df[timestamp_col].dt.hour
features['minute'] = df[timestamp_col].dt.minute
features['week'] = df[timestamp_col].dt.isocalendar().week
features['quarter'] = df[timestamp_col].dt.quarter

# Cyclical encoding (preserves circular nature)
features['hour_sin'] = np.sin(2 * np.pi * features['hour'] / 24)
features['hour_cos'] = np.cos(2 * np.pi * features['hour'] / 24)
features['day_sin'] = np.sin(2 * np.pi * features['dayofweek'] / 7)
features['day_cos'] = np.cos(2 * np.pi * features['dayofweek'] / 7)
features['month_sin'] = np.sin(2 * np.pi * features['month'] / 12)
features['month_cos'] = np.cos(2 * np.pi * features['month'] / 12)

# Binary indicators
features['is_weekend'] = (features['dayofweek'] >= 5).astype(int)
features['is_month_start'] = df[timestamp_col].dt.is_month_start.astype(int)
features['is_month_end'] = df[timestamp_col].dt.is_month_end.astype(int)
features['is_quarter_start'] = df[timestamp_col].dt.is_quarter_start.astype(int)
features['is_quarter_end'] = df[timestamp_col].dt.is_quarter_end.astype(int)

# ============ 2. LAG FEATURES ============

for lag in lag_periods:
    features[f'lag_{lag}'] = df[value_col].shift(lag)

# ============ 3. ROLLING STATISTICS ============

for window in rolling_windows:
    # Central tendency
    features[f'rolling_mean_{window}'] = df[value_col].rolling(window).mean()
    features[f'rolling_median_{window}'] = df[value_col].rolling(window).median()

    # Dispersion
    features[f'rolling_std_{window}'] = df[value_col].rolling(window).std()
    features[f'rolling_var_{window}'] = df[value_col].rolling(window).var()
    features[f'rolling_range_{window}'] = (
        df[value_col].rolling(window).max() - df[value_col].rolling(window).min()
    )

    # Extremes
    features[f'rolling_min_{window}'] = df[value_col].rolling(window).min()
    features[f'rolling_max_{window}'] = df[value_col].rolling(window).max()
```

```
    # Quantiles
    features[f'rolling_q25_{window}'] = df[value_col].rolling(window).quantile(0.25)
    features[f'rolling_q75_{window}'] = df[value_col].rolling(window).quantile(0.75)

# ============ 4. DIFFERENCE FEATURES ============

features['diff_1'] = df[value_col].diff(1)
features['diff_2'] = df[value_col].diff(2)
features['diff_7'] = df[value_col].diff(7)

# Percentage change
features['pct_change_1'] = df[value_col].pct_change(1)
features['pct_change_7'] = df[value_col].pct_change(7)

# ============ 5. EXPONENTIAL MOVING AVERAGES ============

for span in [3, 7, 14, 30]:
    features[f'ema_{span}'] = df[value_col].ewm(span=span).mean()
    features[f'ema_std_{span}'] = df[value_col].ewm(span=span).std()

# ============ 6. AUTOCORRELATION FEATURES ============

for lag in [1, 7, 14]:
    # Rolling correlation with lag
    features[f'autocorr_{lag}'] = df[value_col].rolling(window=30).apply(
        lambda x: x.autocorr(lag=lag) if len(x) > lag else np.nan
    )

# ============ 7. FOURIER FEATURES ============

# Extract dominant frequencies
if len(df) >= 50:
    # Perform FFT
    fft_vals = fft.fft(df[value_col].fillna(method='ffill'))
    fft_freq = fft.fftfreq(len(df))

    # Get top 3 frequencies
    power = np.abs(fft_vals)
    top_freqs_idx = np.argsort(power)[-4:-1]  # Exclude DC component

    for i, idx in enumerate(top_freqs_idx):
        freq = abs(fft_freq[idx])
        if freq > 0:
            features[f'fourier_sin_{i}'] = np.sin(2 * np.pi * freq * np.arange(len(df)))
            features[f'fourier_cos_{i}'] = np.cos(2 * np.pi * freq * np.arange(len(df)))
```

```python
# ============ 8. INTERACTION FEATURES ============

# Value relative to recent statistics
features['value_vs_mean_7'] = df[value_col] / features['rolling_mean_7']
features['value_vs_median_7'] = df[value_col] / features['rolling_median_7']
features['value_vs_max_7'] = df[value_col] / features['rolling_max_7']

# Distance from moving average
features['dist_from_ma_7'] = df[value_col] - features['rolling_mean_7']
features['dist_from_ma_30'] = df[value_col] - features['rolling_mean_30']

# ============ 9. TREND FEATURES ============

# Linear trend over window
def rolling_trend(series, window=14):
    """Fit linear trend in rolling window"""
    trends = []
    for i in range(len(series)):
        if i < window:
            trends.append(np.nan)
        else:
            y = series.iloc[i-window:i].values
            x = np.arange(window)
            if len(y) == window and not np.isnan(y).any():
                slope = np.polyfit(x, y, 1)[0]
                trends.append(slope)
            else:
                trends.append(np.nan)
    return trends

features['trend_7'] = rolling_trend(df[value_col], window=7)
features['trend_14'] = rolling_trend(df[value_col], window=14)

# ============ 10. TARGET ENCODING (if categorical available) ============

# If you have categorical features (e.g., day of week)
# Calculate average target for each category
for col in ['dayofweek', 'hour', 'month']:
    if col in features:
        target_mean = df.groupby(features[col])[value_col].mean()
        features[f'{col}_target_mean'] = features[col].map(target_mean)

print(f"\nCreated {len(features.columns)} features:")
print(f"  - Temporal: ~20")
print(f"  - Lags: {len(lag_periods)}")
print(f"  - Rolling stats: ~{len(rolling_windows) * 7}")
```

```
    print(f"  - Differences: 5")
    print(f"  - EMAs: {4 * 2}")
    print(f"  - Autocorrelations: 3")
    print(f"  - Fourier: ~6")
    print(f"  - Interactions: 5")
    print(f"  - Trends: 2")
    print(f"  - Target encoding: ~3")

    return features

# Usage
df = pd.DataFrame({
    'timestamp': pd.date_range('2024-01-01', periods=1000, freq='H'),
    'value': np.random.randn(1000).cumsum() + 100
})

features = create_time_series_features(df)
print(f"\nShape: {features.shape}")
print(f"\nFirst few features:\n{features.head()}")
```

Key principles: 1. **Temporal:** Capture cyclical patterns with sin/cos 2. **Lags:** Recent history matters 3. **Rolling:** Local statistics 4. **Fourier:** Periodic components 5. **Interactions:** Relative measures 6. **Domain-specific:** Add features based on problem context

---

Q23: Improved Anomaly Detection

```
import numpy as np
import pandas as pd
from scipy import stats

class AnomalyDetector:
    """
    Multiple anomaly detection methods for time series
    """

    def __init__(self, contamination=0.01):
        self.contamination = contamination

    def zscore_static(self, data, threshold=3):
        """
        Static z-score (problematic for non-stationary)
        """
        z = (data - data.mean()) / data.std()
        return np.abs(z) > threshold
```

```
def zscore_rolling(self, data, window=50, threshold=3):
    """
    Rolling z-score - adapts to local statistics
    Better for non-stationary data
    """
    rolling_mean = data.rolling(window=window, center=True).mean()
    rolling_std = data.rolling(window=window, center=True).std()

    z_scores = (data - rolling_mean) / rolling_std
    return np.abs(z_scores) > threshold

def iqr_method(self, data, k=1.5):
    """
    Interquartile range method
    More robust to outliers than z-score
    """
    Q1 = data.quantile(0.25)
    Q3 = data.quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - k * IQR
    upper_bound = Q3 + k * IQR

    return (data < lower_bound) | (data > upper_bound)

def iqr_rolling(self, data, window=50, k=1.5):
    """
    Rolling IQR - adapts to local distribution
    """
    rolling_q25 = data.rolling(window=window, center=True).quantile(0.25)
    rolling_q75 = data.rolling(window=window, center=True).quantile(0.75)
    rolling_iqr = rolling_q75 - rolling_q25

    lower = rolling_q25 - k * rolling_iqr
    upper = rolling_q75 + k * rolling_iqr

    return (data < lower) | (data > upper)

def seasonal_anomaly(self, data, period=24):
    """
    Detect anomalies considering seasonality

    Deseasonalize, then detect anomalies in residuals
    """
    # Calculate seasonal averages
    seasonal_avg = np.zeros(len(data))
```

```
        for i in range(period):
            season_vals = data[i::period]
            seasonal_avg[i::period] = season_vals.mean()

        # Deseasonalized data
        deseasonalized = data - seasonal_avg

        # Detect anomalies in deseasonalized
        return self.zscore_rolling(pd.Series(deseasonalized), window=period*2)

    def mad_method(self, data, threshold=3.5):
        """
        Median Absolute Deviation
        More robust than standard deviation

        MAD = median(|X_i - median(X)|)
        Modified z-score = 0.6745 * (X_i - median(X)) / MAD
        """
        median = data.median()
        mad = np.median(np.abs(data - median))

        if mad == 0:
            return pd.Series([False] * len(data), index=data.index)

        modified_z = 0.6745 * (data - median) / mad
        return np.abs(modified_z) > threshold

    def isolation_forest(self, data, contamination=0.01):
        """
        Isolation Forest for anomaly detection
        """
        from sklearn.ensemble import IsolationForest

        # Reshape for sklearn
        X = data.values.reshape(-1, 1)

        iso_forest = IsolationForest(
            contamination=contamination,
            random_state=42,
            n_estimators=100
        )

        predictions = iso_forest.fit_predict(X)
        anomalies = predictions == -1

        return anomalies
```

```python
def detect_contextual_anomalies(self, data, window=50):
    """
    Contextual anomalies - unusual in local context

    Compare point to local distribution
    """
    anomalies = np.zeros(len(data), dtype=bool)

    for i in range(window, len(data)):
        # Local window
        local_data = data[i-window:i]
        local_mean = local_data.mean()
        local_std = local_data.std()

        # Current point
        current = data.iloc[i]

        # Z-score in local context
        if local_std > 0:
            z = abs((current - local_mean) / local_std)
            if z > 3:
                anomalies[i] = True

    return anomalies

def ensemble_detection(self, data, methods=['rolling_z', 'iqr', 'mad'], vote_threshold=2
    """
    Ensemble multiple methods - vote-based
    Anomaly if >= vote_threshold methods agree
    """
    results = {}

    if 'rolling_z' in methods:
        results['rolling_z'] = self.zscore_rolling(data).astype(int)

    if 'iqr' in methods:
        results['iqr'] = self.iqr_rolling(data).astype(int)

    if 'mad' in methods:
        results['mad'] = self.mad_method(data).astype(int)

    if 'isolation' in methods:
        results['isolation'] = self.isolation_forest(data).astype(int)

    # Vote
```

```
        votes = pd.DataFrame(results).sum(axis=1)
        anomalies = votes >= vote_threshold

        return anomalies, results

    def evaluate_detection(self, detected, true_anomalies):
        """
        Evaluate anomaly detection performance
        """
        from sklearn.metrics import (precision_score, recall_score,
                                     f1_score, confusion_matrix)

        precision = precision_score(true_anomalies, detected)
        recall = recall_score(true_anomalies, detected)
        f1 = f1_score(true_anomalies, detected)

        tn, fp, fn, tp = confusion_matrix(true_anomalies, detected).ravel()

        return {
            'precision': precision,
            'recall': recall,
            'f1_score': f1,
            'true_positives': tp,
            'false_positives': fp,
            'false_negatives': fn,
            'true_negatives': tn
        }

# Example usage
np.random.seed(42)
n = 1000
t = np.arange(n)

# Normal data with seasonality
seasonal = 10 * np.sin(2 * np.pi * t / 50)
trend = 0.01 * t
noise = np.random.normal(0, 1, n)
data = pd.Series(trend + seasonal + noise)

# Inject anomalies
true_anomalies = np.zeros(n, dtype=bool)
anomaly_indices = [100, 250, 500, 750, 900]
for idx in anomaly_indices:
    data.iloc[idx] += 20  # Large spike
    true_anomalies[idx] = True
```

```python
# Detect anomalies
detector = AnomalyDetector()

# Single methods
z_anomalies = detector.zscore_rolling(data)
iqr_anomalies = detector.iqr_rolling(data)
mad_anomalies = detector.mad_method(data)

# Ensemble
ensemble_anomalies, method_results = detector.ensemble_detection(
    data,
    methods=['rolling_z', 'iqr', 'mad'],
    vote_threshold=2
)

# Evaluate
results = detector.evaluate_detection(ensemble_anomalies, true_anomalies)
print("\nEnsemble Detection Performance:")
for metric, value in results.items():
    print(f"  {metric}: {value}")
```

Key improvements over simple z-score: 1. **Rolling statistics** - adapts to non-stationarity 2. **MAD** - robust to outliers 3. **IQR** - distribution-free 4. **Seasonal awareness** - accounts for patterns 5. **Ensemble** - combines multiple methods 6. **Contextual** - local vs global anomalies

---

Q30: Comparative Analysis Framework

```python
import numpy as np
import pandas as pd
from sklearn.metrics import mean_squared_error, mean_absolute_error
from scipy import stats
import time

class TimeSeriesComparison:
    """
    Framework for fair comparison of forecasting methods
    """

    def __init__(self, train_size=0.8, forecast_horizon=12):
        self.train_size = train_size
        self.forecast_horizon = forecast_horizon
        self.results = {}

    def train_test_split(self, data):
        """Chronological split"""
```

```
        split_idx = int(len(data) * self.train_size)
        train = data[:split_idx]
        test = data[split_idx:]
        return train, test

    def evaluate_model(self, model_name, model, train, test):
        """
        Evaluate single model with comprehensive metrics
        """
        print(f"\nEvaluating {model_name}...")

        # Fit
        start_time = time.time()
        model.fit(train)
        fit_time = time.time() - start_time

        # Predict
        start_time = time.time()
        forecast = model.predict(steps=len(test))
        predict_time = time.time() - start_time

        # Metrics
        mse = mean_squared_error(test, forecast)
        rmse = np.sqrt(mse)
        mae = mean_absolute_error(test, forecast)
        mape = np.mean(np.abs((test - forecast) / test)) * 100

        # Residuals
        residuals = test - forecast

        # Store results
        self.results[model_name] = {
            'RMSE': rmse,
            'MAE': mae,
            'MAPE': mape,
            'fit_time': fit_time,
            'predict_time': predict_time,
            'forecast': forecast,
            'residuals': residuals
        }

        return forecast

    def compare_all(self, models_dict, data):
        """
        Compare all models with same data splits
```

```
        models_dict: {'ARIMA': arima_model, 'LSTM': lstm_model, ...}
        """
        train, test = self.train_test_split(data)

        for name, model in models_dict.items():
            self.evaluate_model(name, model, train, test)

        return self.create_summary()

    def create_summary(self):
        """Create comparison table"""
        summary = pd.DataFrame(self.results).T
        summary = summary[['RMSE', 'MAE', 'MAPE', 'fit_time', 'predict_time']]

        # Rank models
        for metric in ['RMSE', 'MAE', 'MAPE']:
            summary[f'{metric}_rank'] = summary[metric].rank()

        return summary.sort_values('RMSE')

    def statistical_significance(self, model1, model2):
        """
        Diebold-Mariano test for forecast comparison

        H0: Two forecasts have equal accuracy
        H1: Different accuracy
        """
        e1 = self.results[model1]['residuals']
        e2 = self.results[model2]['residuals']

        # Loss differential
        d = e1**2 - e2**2

        # Mean loss differential
        d_mean = d.mean()

        # Variance of loss differential (with autocorrelation correction)
        # Newey-West standard error
        n = len(d)
        gamma_0 = ((d - d_mean) ** 2).sum() / n

        # Simplified (full version needs HAC correction)
        var_d = gamma_0 / n

        # Test statistic
```

```
        dm_stat = d_mean / np.sqrt(var_d)

        # P-value (two-tailed)
        p_value = 2 * (1 - stats.norm.cdf(abs(dm_stat)))

        return {
            'DM_statistic': dm_stat,
            'p_value': p_value,
            'significant': p_value < 0.05,
            'better_model': model1 if dm_stat < 0 else model2
        }

    def visualize_comparison(self, data):
        """
        Create comprehensive visualization
        """
        import matplotlib.pyplot as plt

        fig, axes = plt.subplots(3, 2, figsize=(15, 12))

        train, test = self.train_test_split(data)
        test_index = range(len(train), len(train) + len(test))

        # 1. Forecasts comparison
        ax = axes[0, 0]
        ax.plot(test_index, test, 'k-', label='Actual', linewidth=2)
        for name in self.results.keys():
            forecast = self.results[name]['forecast']
            ax.plot(test_index, forecast, label=name, alpha=0.7)
        ax.set_title('Forecast Comparison')
        ax.legend()
        ax.grid(True, alpha=0.3)

        # 2. Metrics comparison (bar chart)
        ax = axes[0, 1]
        metrics_df = self.create_summary()[['RMSE', 'MAE', 'MAPE']]
        metrics_df.plot(kind='bar', ax=ax)
        ax.set_title('Metrics Comparison')
        ax.set_ylabel('Value')

        # 3. Residuals distribution
        ax = axes[1, 0]
        for name in self.results.keys():
            residuals = self.results[name]['residuals']
            ax.hist(residuals, alpha=0.5, bins=30, label=name)
        ax.set_title('Residuals Distribution')
```

```
        ax.set_xlabel('Residual')
        ax.legend()

        # 4. Residuals Q-Q plot (first model)
        ax = axes[1, 1]
        first_model = list(self.results.keys())[0]
        residuals = self.results[first_model]['residuals']
        stats.probplot(residuals, dist="norm", plot=ax)
        ax.set_title(f'Q-Q Plot ({first_model})')

        # 5. Computation time
        ax = axes[2, 0]
        times_df = self.create_summary()[['fit_time', 'predict_time']]
        times_df.plot(kind='bar', ax=ax)
        ax.set_title('Computation Time')
        ax.set_ylabel('Seconds')

        # 6. Error over time
        ax = axes[2, 1]
        for name in self.results.keys():
            residuals = self.results[name]['residuals']
            ax.plot(abs(residuals), label=name, alpha=0.7)
        ax.set_title('Absolute Error Over Time')
        ax.set_xlabel('Time Step')
        ax.set_ylabel('|Error|')
        ax.legend()
        ax.grid(True, alpha=0.3)

        plt.tight_layout()
        return fig

# Usage example
if __name__ == "__main__":
    # Generate synthetic data
    np.random.seed(42)
    n = 200
    t = np.arange(n)
    trend = 0.5 * t
    seasonal = 10 * np.sin(2 * np.pi * t / 12)
    noise = np.random.normal(0, 2, n)
    data = pd.Series(trend + seasonal + noise)

    # Define models (pseudo-code - replace with actual models)
    class DummyARIMA:
        def fit(self, train): self.mean = train.mean()
        def predict(self, steps): return np.full(steps, self.mean)
```

```
class DummyLSTM:
    def fit(self, train): self.mean = train.mean() + 1
    def predict(self, steps): return np.full(steps, self.mean)

models = {
    'ARIMA': DummyARIMA(),
    'LSTM': DummyLSTM()
}

# Compare
comparison = TimeSeriesComparison(train_size=0.8, forecast_horizon=12)
summary = comparison.compare_all(models, data)

print("\n" + "="*60)
print("COMPARATIVE ANALYSIS RESULTS")
print("="*60)
print(summary)

# Statistical test
dm_result = comparison.statistical_significance('ARIMA', 'LSTM')
print("\n" + "="*60)
print("DIEBOLD-MARIANO TEST (ARIMA vs LSTM)")
print("="*60)
for key, value in dm_result.items():
    print(f"{key}: {value}")
```

Key elements for fair comparison: 1. **Same data split** - chronological, no overlap 2. **Same metrics** - RMSE, MAE, MAPE minimum 3. **Same hyperparameter budget** - e.g., 50 trials each 4. **Statistical testing** - Diebold-Mariano for significance 5. **Multiple dimensions** - accuracy, speed, memory, interpretability 6. **Reproducibility** - fixed seeds, documented environments

---

Key Preparation Areas

Mathematics: - Stationarity, autocorrelation, decomposition - ARIMA theory (AR, MA, integration) - Fourier analysis, spectral methods - Statistical hypothesis testing - Optimization (gradient descent, MLE)

Python Libraries: - NumPy/SciPy: numerical computing - Pandas: time series manipulation - Statsmodels: ARIMA, seasonal decomposition - Scikit-learn: ML models, validation - TensorFlow/PyTorch: deep learning

Time Series Concepts: - Trend, seasonality, noise - Stationarity and transformations - Autocorrelation functions - Forecast intervals and uncertainty - Multi-step ahead forecasting

Research Skills: - Literature review methodology - Experimental design - Result documentation - Scientific writing - Reproducibility practices

Resources

Books: - "Forecasting: Principles and Practice" by Hyndman & Athanasopoulos - "Time Series Analysis and Its Applications" by Shumway & Stoffer

Online: - Statsmodels documentation - Kaggle time series tutorials - Papers with Code (time series section)

Practice: - Implement ARIMA from scratch - Work through statsmodels examples - Kaggle time series competitions - Replicate results from papers

Interview Strategy

Behavioral: - Emphasize research experience - Show independence and initiative - Demonstrate clear communication - Highlight documentation skills

Technical: - Show mathematical understanding - Implement from scratch when asked - Discuss trade-offs and assumptions - Connect theory to practice - Be honest about limitations

Remember: This is a research-focused role. Show curiosity, rigor, and ability to work independently while collaborating weekly.

Good luck!